

클로드 코드 토큰 최적화 대화 전체 기록

(첨부 문서 전문 + 대화 내용 전체 — 수정/축약 없이 원문 그대로 정리)

Part 1. 첨부 파일 전문 — 유튜브 영상 분석 보고서

제공해주신 유튜브 영상(클로드 코드 한도, 이거 모르면 계속 낭비하게됩니다 (feat. 클로드 공식 블로그))의 전체 내용을 상세히 분석 및 검증한 보고서입니다.

전반부 핵심 요약

- **제작 목적:** 엔트로픽(Anthropic) 공식 기술 블로그에서 다룬 LLM 에이전트 비용 계산 메커니즘을 토대로, 사용자가 클로드 코드(Claude Code) 등 대화형 코딩 에이전트를 운용할 때 사용 한도 초과 및 과도한 비용 소모가 발생하는 기술적 원인을 규명하고 실천 최적화 방안을 공유하기 위함입니다.
- **핵심 메시지:** 에이전트 사용 비용은 단순 입력 텍스트 양이 아닌 'GPU 연산 점유 시간'에 비례합니다. 매 대화 턴(Turn)마다 과거 컨텍스트 전체가 재전송되는 구조적 특성 때문에, 프롬프트 캐싱의 보존 여부, 세션 길이, 불필요한 파일/도구 적재 여부가 비용과 사용 한도에 결정적 영향을 미칩니다.
- **최종 결론:** 비용 낭비의 핵심 요인은 '장기 세션 누적', '과도한 컨텍스트 적재', '작업 대비 과도한 모델 및 깊은 사고(Effort) 설정', '캐시 무효화' 순으로 나타납니다. 세션 분리(/clear), 요약 압축(/compact), 히스토리 롤백(/rewind), 파일 멘션(@), 서브 에이전트 분할을 적절히 결합해 컨텍스트를 가볍게 유지해야 요금 한도 내에서 작업량을 극대화할 수 있습니다.

타임스탬프별 상세 분석

[00:00] 에이전트 도구의 비용 부과 방식과 GPU 연산 시간의 상관관계

- **핵심 주장:** 클로드 코드와 같은 에이전트 도구는 동일한 작업이라도 운용 방식에 따라 토큰 및 비용 차이가 극단적으로 갈립니다. 요금표의 토큰 단위는 본질적으로 'GPU 추론 시간'을 환산한 것입니다.
- **구체적 근거 및 데이터:**
 - 전기요금 청구서에 사용 전력량(kWh)이 표시되지만 배후 실비용은 발전소 연료 소비량인 것과 동일하게, LLM 요금은 100 만 토큰당 단가로 표시되지만 실비용은 고비용 GPU 가 토큰을 처리하는 연산 시간에 대응합니다.

- 토큰 처리 시간을 결정하는 3 요소는 '모델 체급(파라미터 수)', '토큰 방향성(입력 vs 출력)', '프롬프트 캐시 적중 여부'입니다.
- **사용된 예시:** 동일 작업이라도 사용자의 프롬프트 구성 및 세션 길이에 따라 한도가 금방 바닥나거나 여유로운 격차가 발생하는 현상을 지적했습니다.

[01:31] 모델 체급에 따른 계산 비용과 단가 승수 구조

- **핵심 주장:** 고성능 모델일수록 내부 파라미터(가중치) 수가 많아 단일 토큰 처리 시 발생하는 연산량과 인건비가 비례하여 상승합니다. 모든 비용 최적화는 모델 기본 단가에 곱연산됩니다.
- **구체적 근거 및 데이터:**
 - 모델 크기는 '개당 단가', 토큰은 '소비 수량'으로 정의됩니다.
 - 이후에 적용되는 모든 세션 절약 팁의 절대적 가치는 베이스 모델의 단가에 비례하여 증폭됩니다.
- **사용된 예시:** 동일한 1 장의 서류를 수백 명이 검토하는 조직과 수만 명이 검토하는 조직의 인건비 차이에 비유하며, 단순 작업에는 경량 모델을, 고난도 추론에는 대형 모델을 배정하는 전략을 제시했습니다.

[02:42] 입력(프리필)과 출력(디코드)의 아키텍처 차이 및 사고(Effort) 설정

- **핵심 주장:** 출력 토큰은 병렬 처리가 가능한 입력 토큰과 달리 순차적 자기회귀(Auto-regressive) 연산이 필요하여 입력 대비 약 5 배가량 단가가 높습니다. 사고(Thinking) 토큰 또한 출력 토큰에 해당합니다.
- **구체적 근거 및 데이터:**
 - 프리필(Prefill, 입력): 시스템 프롬프트, CLAUDE.md, 이전 대화 및 읽어 들인 파일 전체를 모델이 답을 내기 전 한 번에 파악하는 단계로 병렬 처리가 가능합니다.
 - 디코드(Decode, 출력): 직전 토큰이 생성되어야 다음 토큰을 연산하는 순차 구조입니다. 200 토큰 출력 시 전체 신경망 모델을 200 회 연달아 반복 구동하므로 GPU 점유 시간이 길어져 입력 대비 5 배 비싸게 책정됩니다.
 - 내부 사고 과정(Thinking)과 도구 호출(Tool call) 명령 또한 출력 토큰으로 분류되어 5 배 단가로 과금됩니다.
 - Effort 파라미터를 상향하면 추론 깊이가 늘어나는 대신 출력 토큰이 급증하며, 이 설정값은 현재 세션 종료 후 다음 세션까지 전역 상태로 유지됩니다.
- **사용된 예시:** 시험지 지문을 전체적으로 빠르게 훑어보는 병렬 읽기(프리필)와, 벽돌 200 장을 한 장씩 200 번 왕복하며 나르는 작업(디코드)의 비유를 들었습니다. 지난 작업

시 상향해 둔 Effort 레벨이 유지되어 일상적인 작업에서 토큰 누수가 발생하는 사례를 경고했습니다.

[06:01] 프롬프트 캐싱(Prompt Caching) 메커니즘과 접두사(Prefix) 불변 규칙

- **핵심 주장:** 직전 요청의 연산 메모리를 재사용하는 프롬프트 캐싱은 단가를 1/10(0.1 배) 수준으로 낮추는 핵심 기능이지만, 접두사 무효화 조건을 모르면 오히려 비용이 증가할 수 있습니다.
- **구체적 근거 및 데이터:**
 - 무상태(Stateless)인 LLM 특성상 대화가 이어질 때마다 전체 컨텍스트를 재전송하므로, 서버는 직전 계산 결과를 메모리에 보관(캐시 쓰기)한 뒤 동일 접두사가 들어오면 재사용(캐시 읽기)합니다.
 - 캐시 저장 비용은 초기 1 회 생성 시 정상가의 최대 2 배(1-hour TTL 기준) 수준이지만, 재사용 시 정상가의 0.1 배(10%)만 부과됩니다.
 - 20 회 대화 턴 발생 시: 비캐싱은 20 의 비용(1×20)이 발생하지만, 캐싱 시 3.9(최초 저장 2 + 이후 19 회 $\times 0.1$)만 발생하여 약 5 배 이상의 비용 절감 효과가 발생합니다.
 - 캐시 검증은 기차 칸처럼 맨 앞부터 순차 비교하므로, 앞부분이 바뀌면 도미노처럼 그 뒤의 모든 구간이 캐시 미스 처리되어 전체 재연산이 발생합니다.
 - 캐시 파괴 요인 3 가지: 대화 도중 모델/Effort 꼬리표(Metadata) 변경, /compact 명령으로 대화 요약본 교체, 마지막 요청 이후 TTL(1 시간) 만료.
- **사용된 예시:** 앞쪽 기관차(도구 정의, 시스템 프롬프트, CLAUDE.md)부터 순서대로 연결된 열차 비유를 제시하며, 중간 화물칸이 바뀌면 뒤편의 모든 연결부가 재조립되어야 함을 설명했습니다.

[10:35] 대화 오류 해결 전략: /compact vs /rewind 선택 기준

- **핵심 주장:** 의도치 않은 탐색이나 오류가 발생했을 때 /compact 를 쓰면 기존 캐시가 전면 파괴되므로, 최근 턴의 단순 오류라면 /rewind 를 사용하는 것이 캐시 유지 면에서 월등히 유리합니다.
- **구체적 근거 및 데이터:**
 - /compact: 대화 전체를 요약본으로 교체하므로 기존 대화 캐시가 소멸되며, 요약 생성 자체에 추가 토큰 비용이 발생합니다.
 - /rewind: 맨 끝에 붙은 대화 턴 몇 개만 잘라내므로, 잘라낸 지점 앞부분의 캐시 데이터는 완벽히 보존되어 재연산 비용이 발생하지 않습니다.

- 자리를 비우기 직전 대화 다이어트가 필요하다면 캐시 만료(1 시간) 전에 미리 /compact 를 수행해야 요약 작업 시 캐시 읽기 할인(0.1 배)을 적용받을 수 있습니다.
- **사용된 예시:** 클로드가 엉뚱한 디렉토리나 대용량 파일을 반복 조회하여 히스토리가 오염된 경우의 해결 패턴을 대조했습니다.

[12:00] 컨텍스트 적재의 눈덩이 효과와 정적 부하(MCP) 최소화

- **핵심 주장:** 세션에 한 번 들어온 모든 데이터(읽은 파일, 터미널 실행 로그)는 세션 종료 시까지 매 턴마다 재전송되며, 이는 비용뿐 아니라 모델의 추론 품질(책상 위의 짐)까지 저하시킵니다.
- **구체적 근거 및 데이터:**
 - 세션 시작 시 CLAUDE.md, 시스템 지침, MCP 도구 명세가 자동으로 적재됩니다. 미사용 MCP 서버가 연결되어 있으면 도구 스키마만으로 수천~수만 토큰이 매 대화마다 누적 소모됩니다.
 - 세션 내에서 /context 명령어를 통해 적재 현황을 주기적으로 모니터링하고 불필요한 MCP 연결을 해제해야 합니다.
- **사용된 예시:** 작업 책상 위에 쓰지 않는 짐이 가득 차 있으면 작업자의 업무 집중도와 처리 속도가 떨어지는 현상에 비유했습니다.

[14:13] 지시 모호성에 따른 토큰 누수와 @ 멘션 최적화

- **핵심 주장:** 사용자의 프롬프트가 모호할수록 클로드는 파일 탐색 및 명령 실행을 남발하여 불필요한 토큰을 대량 생성합니다. 명확한 파일 지정과 @ 멘션을 통해 탐색 단계를 원천 차단해야 합니다.
- **구체적 근거 및 데이터:**
 - "뭔가 안 돼" 식의 질문: 파일 검색, 디렉토리 탐색, 연관 파일 열람 등 대량의 읽기 토큰과 터미널 출력 토큰이 세션 히스토리에 영구 적재됩니다.
 - 파일명 텍스트 명시: 에이전트의 불필요한 탐색 과정을 생략하고 해당 파일 1 회 조회로 단축합니다.
 - @ 멘션 파일 첨부: 사용자 메시지 전송 시점에 파일 내용이 선행 포함되므로, 에이전트의 "도구 실행 요청 -> 결과 반환"이라는 1 회 왕복 단계(도구 호출 출력 토큰 및 실행 지연) 자체가 제거됩니다.
 - 단, 이미 세션에 추가된 파일을 중복해서 @ 멘션하면 동일 파일 사본이 추가 적재되어 토큰이 낭비되므로 두 번째 턴부터는 파일명만 언급해야 합니다.

- **사용된 예시:** 골뱅이(@) 기호 자동완성을 통해 대상 소스 코드를 직접 찍어서 프롬프트에 포함시키는 실무 워크플로우를 제시했습니다.

[15:29] 세션 수명 주기 관리: 긴 단일 세션 vs 짧은 분할 세션

- **핵심 주장:** 40 번째 대화에서는 이전 39 번의 전체 대화 내역을 모두 다시 읽어야 하므로, 하나의 긴 세션을 유지하는 것은 세션을 목적별로 분할하는 것보다 기하급수적으로 비쌉니다.
- **구체적 근거 및 데이터:**
 - /clear: 이전 맥락을 완전히 비우고 초기화하여 독립적인 작업으로 새 출발할 때 사용합니다.
 - /compact [지침]: 작업 흐름은 유지하되 앞선 시행착오를 건너낼 때 사용하며, "결정된 아키텍처와 수정 파일 경로만 남겨줘"와 같이 압축 대상 기준을 직접 명시할 수 있습니다.
- **사용된 예시:** 블로그 글 정리를 마친 뒤 전혀 다른 코딩 작업을 이어갈 때 /clear 를 치지 않고 대화를 지속하여 과거 글 데이터가 계속 과금되는 비효율을 언급했습니다.

[17:09] 서브 에이전트(Sub-agent)를 통한 컨텍스트 오염 격리

- **핵심 주장:** 대량의 파일을 읽고 검색해야 하는 작업은 메인 세션이 아닌 별도의 격리된 서브 에이전트에게 위임하여 메인 컨텍스트의 오염을 방지해야 합니다.
- **구체적 근거 및 데이터:**
 - 서브 에이전트는 기본 시스템 프롬프트 및 환경 설정만 상속받고 기존 대화 히스토리는 배제한 백지 상태에서 시작합니다.
 - 작업 과정에서 읽은 대용량 로그, 중간 파일 내용은 작업 완료 즉시 소멸하며 메인 세션에는 최종 요약 결론 1 줄만 보고됩니다.
 - 단, 메인 대화와 격리되어 있으므로 기존 파일을 중복 조회할 수 있고 자체 연산 토큰 비용이 들기 때문에, 단순 질의가 아닌 '조사 분량은 방대하나 최종 보관할 가치는 결론 한 줄에 불과한 작업'에 한해 경제적입니다.
- **사용된 예시:** 방대한 서류 더미에서 결함 1 건을 찾아 팀장(메인 세션)에게 1 장짜리 보고서로 제출하는 직원의 역할 분담 구조를 설명했습니다.

[19:36] 엔트로픽 공식 비용 영향도 랭킹 4 단계 요약

- **핵심 주장:** 전체 에이전트 비용 지출에 가장 치명적인 영향을 주는 요소는 순서대로 긴 세션, 컨텍스트 과적, 과도한 모델/Effort, 캐시 파괴입니다.

- **구체적 근거 및 데이터:**

- 1 위: 장기 누적 세션 (대화 전체 재전송으로 토큰 제공 비례 증가)
- 2 위: 컨텍스트 과적 (미사용 MCP, 불필요 파일 적재, 터미널 과다 출력)
- 3 위: 작업 난이도 대비 과도한 모델 및 Effort 상향 방치
- 4 위: 중간 설정 변경 및 1 시간 이상 유휴로 인한 프롬프트 캐시 파괴

- **사용된 예시:** 기술 블로그 원문 맺음말에 수록된 공식 우선순위를 역순으로 점검하는 체크리스트를 전달했습니다.

핵심 인사이트

- **컨텍스트 엔지니어링의 본질:** 에이전트 환경에서 컨텍스트는 단순한 작업 메모리가 아니라 '매 턴마다 비용을 발생시키는 과금 단위'이자 모델의 추론 정확도를 떨어뜨리는 노이즈 원천입니다. 즉, 컨텍스트를 비우는 작업이 최우선 개발 역량입니다.
- **접두사(Prefix) 안정성 확보:** 프롬프트 캐시는 완벽한 선두 일치(Prefix Match) 기반으로 동작하므로, 시스템 프롬프트 변경, MCP 도구 동적 추가, 세션 중간 모델 스위칭 등 선두 구조를 흔드는 행동을 개발 파이프라인에서 원천 배제해야 합니다.
- **작업의 분리와 격리(Divide and Conquer):** 단일 세션에서 설계, 탐색, 디버깅, 구현을 원스톱으로 처리하려는 습관을 버리고, 로그 분석이나 탐색 단계는 서브 에이전트 또는 스크립트로 분리하여 메인 세션의 수명을 짧게 유지하는 모듈화가 필요합니다.

사실 여부 및 신뢰성 검증

입력 토큰 대비 출력 토큰 5 배 단가 주장 (사실)

- 엔트로픽 공식 가격 정책에 따르면 Claude 3.5 Sonnet 및 Sonnet 4.6(입력 \$3 / 출력 \$15 per MTok), Opus 계열(입력 \$5 / 출력 \$25 per MTok), Haiku 계열(입력 \$1 / 출력 \$5 per MTok) 등 주력 모델 전반에서 출력 단가는 입력 단가의 정확히 5 배로 책정되어 있어 발화자의 설명은 사실에 부합합니다.

프롬프트 캐싱 단가 구조 및 1 시간 만료 기준 (부분 정밀화 필요)

- **영상 주장:** 캐시 저장 시 최대 2 배가 들고 재사용 시 0.1 배가 들며, 1 시간이 지나면 만료된다고 일괄 설명했습니다.
- **검증 및 보완:**
 - 엔트로픽의 기본 프롬프트 캐싱 수명(TTL)은 5 분입니다. 5 분 TTL 의 경우 캐시 쓰기 비용은 기본 입력가의 1.25 배(25% 할증), 캐시 읽기는 0.1 배(90% 할인)입니다.

- 엔트로픽은 추가 설정(ttl: "1h")을 통해 1 시간 TTL 옵션을 제공하며, 이때 캐시 쓰기 비용이 기본 입력가의 2.0 배로 부과됩니다.
- 클로드 코드 내부 구현에서는 백그라운드에서 1 시간 단위 캐시를 기본으로 활용하거나 연장 갱신(Refresh) 정책을 취하고 있으나, 일반 API 관점에서는 5 분 TTL(1.25 배)과 1 시간 TTL(2.0 배)이 구분되어 있으므로 영상의 "저장 시 2 배, 1 시간 유지" 설명은 1 시간 TTL 기준을 기본 전제로 단순화한 설명입니다.

대화 중간 모델/Effort 변경 시 캐시 무효화 (사실)

- 프롬프트 캐시의 키 값(Cache Key)에는 모델 식별자 및 추론 관련 하이퍼파라미터 메타데이터가 포함됩니다. 따라서 텍스트가 100% 동일하더라도 모델이나 Effort 레벨을 전환하면 캐시 미스가 발생하여 전체 재계산이 이루어집니다.

서브 에이전트의 컨텍스트 격리 구조 (사실)

- Claude Code 아키텍처에서 서브 에이전트(Task Agent)는 메인 대화 세션의 히스토리 버퍼를 공유받지 않고 별도의 빈 실행 환경에서 실행되며, 작업 완료 후 최종 반환값만 메인 스레드로 주입되는 샌드박스 패턴을 사용하므로 발화 내용이 기술적으로 정확합니다.

YouTube video views will be stored in your YouTube History, and your data will be stored and used by YouTube according to its Terms of Service

Part 2. 대화 기록

[사용자 질문 1]

이 내용이 맞는지와 클로드 코드로 커다란 프로젝트를 지속적으로 개발하고 있는데(GD RDM 프로젝트) 토큰 사용을 최적화 하는 방법을 좀 알려줘. 요즘 내가 회사에서 사용하는 클로드 엔터프라이즈 버전이 너무 토큰 소모가 심한거 같아

[※ 첨부: 유튜브 영상 분석 보고서 txt 파일 — 본 문서 상단 "첨부 파일 전문" 참조]

[Claude 답변 1]

영상 내용 검증

영상의 핵심 메커니즘 설명은 전반적으로 정확합니다. 다만 프롬프트 캐싱 TTL 부분은 보완이 필요합니다.

항목	영상 설명	실제 (Anthropic 공식)
기본 캐시 TTL	"1시간" 전제로 설명	기본값은 5분입니다
5분 캐시 쓰기 비용	언급 안 함	기본 입력가의 1.25배
1시간 캐시 쓰기 비용	"최대 2배"	정확히 2배 (별도 옵션 설정 시)
캐시 읽기 비용	"0.1배"	0.1배 (정확)
출력 토큰이 입력의 5배	정확	정확 (Sonnet 기준 입력 \$3 / 출력 \$15 등)

즉 "저장 시 2배, 1시간 유지"라는 설명은 1시간 TTL 옵션을 켜고 있을 때만 해당하고, 기본 동작은 5분 TTL + 1.25배 할증입니다. 다만 Claude Code가 세션 중에는 백그라운드에서 캐시를 자주 갱신(refresh)하기 때문에, 5분 이내로 대화가 이어지는 한 캐시가 계속 살아있다는 실질적 결론 자체는 맞습니다. 서버에이전트 컨텍스트 격리, /compact vs /rewind 차이, 캐시가 접두사(prefix) 기준으로 깨진다는 내용은 모두 정확합니다.

GD RDM 같은 대형 프로젝트에서의 실전 최적화

CSV 스키마가 크고 세션이 길어지는 프로젝트 특성상 아래 순서로 적용하시면 효과가 큼니다.

1) 세션을 수명 단위로 끊기

- 스키마 분석, 검증 로직 작성, 버그 수정처럼 목적이 바뀌는 시점마다 /clear 로 새로 시작하세요. 40 번째 턴은 앞선 39 개 턴 전체를 다시 읽는 구조라, 긴 세션 하나를 붙잡고 있는 것이 가장 비쌉니다.
- 같은 작업 흐름 안에서 시행착오만 걷어내고 싶다면 "/compact 결정된 스키마 규칙과 수정 파일 경로만 남겨줘"처럼 압축 기준을 직접 지정하세요. 자동 컴팩션(auto-

compact)이 걸리는 시점은 모델이 가장 "정신없는" 상태이므로, 컨텍스트가 40%를 넘기 전에 수동으로 끊는 걸 권장합니다.

2) 잘못된 시도는 /compact가 아니라 /rewind로

- 단순 실수(영동한 파일을 읽었다, 접근이 틀렸다)는 /rewind 로 되돌리세요. /compact 는 캐시를 통째로 무효화시키지만, /rewind 는 잘라낸 지점 앞의 캐시를 그대로 보존합니다.

3) 542컬럼 스키마 같은 대용량 데이터는 서버에이전트로 격리

- CSV 전체를 훑거나 로그를 대량 검색해야 하는 작업은 메인 세션이 아니라 서버에이전트에게 위임하세요. 탐색 과정에서 읽은 수천 줄은 서버에이전트 안에서 소멸하고, 메인 세션에는 결론 요약만 돌아옵니다.

4) 파일은 애매하게 찾게 하지 말고 @ 멘션으로 직접 지정

- "스키마 관련 파일 찾아서 수정해줘" 같은 모호한 지시는 탐색 왕복 비용을 유발합니다. @rdm_schema.csv 처럼 직접 지정하면 탐색 단계가 통째로 사라집니다. 단, 같은 파일을 매 턴 반복 멘션하면 중복 적재되니 두 번째 턴부터는 파일명만 텍스트로 언급하세요.

5) 불필요한 MCP 연결 해제 + CLAUDE.md 슬림화

- /context 명령으로 현재 적재량을 주기적으로 확인하세요. 안 쓰는 MCP 서버가 연결돼 있으면 도구 스펙만으로 매 턴 수천 토큰이 누적됩니다. CLAUDE.md 는 200 줄 이내로 유지하고, 특수 워크플로우는 스킬(skill)로 분리해 필요할 때만 로드되게 하세요.

6) 작업 난이도에 맞게 모델·Effort 낮추기

- 단순 리팩터링·포매팅에는 가벼운 모델을, 복잡한 아키텍처 판단에만 상위 모델을 쓰세요. Effort 를 한 번 올리면 다음 세션까지 전역 설정으로 남아있는 경우가 많으니, 작업이 끝나면 다시 낮춰두는 습관이 필요합니다.

참고: 회사에서 쓰시는 Enterprise 버전은 API 종량제가 아니라 좌석당 사용량 한도(rate limit) 방식이지만, 한도 소모량을 결정하는 내부 메커니즘(캐시 적중률, 컨텍스트 크기)은 API와 동일합니다. 그래서 위 방법들이 "한도가 빨리 차는" 문제 해결에도 그대로 적용됩니다.

[사용자 질문 2]

목적이 바뀌는 시점마다 /clear로 새로 시작하다가 혹시 앞의 내용이 필요한 경우 다시 해당 내용을 읽어야 할때 토큰이 더 소모되는거 아냐?

[Claude 답변 2]

네, 정확한 우려이십니다. 다만 두 비용의 성격이 다르다는 점이 핵심입니다.

왜 그럼에도 /clear 가 유리한가

- **세션을 유지하는 비용:** 이전 대화 전체가 매 턴마다 반복 재전송됩니다. 턴이 늘어날수록 누적되는 구조라, 필요 없어진 시행착오·탐색 로그까지 계속 끌고 다니게 됩니다.
- **/clear 후 다시 불러오는 비용:** 필요한 시점에 딱 한 번 필요한 부분만 다시 읽으면 됩니다.

즉 "전체를 매번 반복해서 내는 비용" vs "필요할 때 한 번만 내는 비용"의 차이입니다. 특히 /clear 전 세션에서 이미 많은 시행착오, 실패한 접근, 겹가지 질문이 쌓여 있었다면 그 전부를 매 턴 다시 내는 것보다, 필요한 결론만 나중에 한 번 불러오는 게 훨씬 쌉니다.

다시 불러올 때도 손해를 줄이는 방법

1) /clear 전에 "인수인계 노트"를 만들어두기

- /clear 하기 직전에 "지금까지 결정된 스키마 규칙, 수정한 파일 경로, 남은 이슈를 정리해줘"라고 요청해서 짧은 요약은 받아두세요.
- 이걸 PROGRESS.md 나 NOTES.md 같은 파일로 저장해두면, 나중에 전체 대화를 다시 훑을 필요 없이 이 요약 파일 하나만 @ 멘션해서 불러오면 됩니다. 원본 대화 전체(수만 토큰)가 아니라 요약본(수백 토큰)만 다시 읽는 셈이라 비용 차이가 큼니다.

2) 정말 필요한 것만 골라서 불러오기

- "전체 대화 다시 봐줘"가 아니라 "GD RDM 스키마 검증 로직 관련 부분만"처럼 구체적으로 지정하면, 불필요한 내용까지 재적재하는 걸 막을 수 있습니다.

3) 판단 기준

- 같은 세션 안에서 왔다 갔다 할 가능성이 높은 작업(예: 같은 버그를 여러 각도로 계속 파고드는 중)이면 /clear 대신 /compact 로 요약만 압축해서 유지하는 게 낫습니다.
- 반대로 완전히 다른 작업으로 넘어가면서 "혹시 몰라서" 이전 내용을 남겨두는 경우라면, 대부분 다시 안 쓰게 되므로 /clear 가 유리합니다. "나중에 필요할 수도" 정도의 막연한 이유로 세션을 계속 유지하는 것이 실제로는 가장 비효율적인 패턴입니다.

[사용자 질문 3]

아래 내용에서 내가 /clear 하기 직전에 "지금까지 결정된 스키마 규칙, 수정한 파일 경로, 남은 이슈를 정리해줘"라고 하면 클로드 코드가 알아서 PROGRESS.md나 NOTES.md 같은 파일로 저장하는 거야 아님 내가 구체적으로 어떤 파일로 저장해 줘 라고 해야 하는거야?

1) /clear 전에 "인수인계 노트"를 만들어두기

- /clear 하기 직전에 "지금까지 결정된 스키마 규칙, 수정한 파일 경로, 남은 이슈를 정리해줘"라고 요청해서 짧은 요약은 받아두세요.

- 이걸 PROGRESS.md 나 NOTES.md 같은 파일로 저장해두면,

[Claude 답변 3]

아니요, 자동으로 파일에 저장되지는 않습니다. "정리해줘"라고만 요청하면 클로드 코드는 그 내용을 채팅 응답(대화 텍스트)으로만 출력합니다. /clear를 하면 그 화면의 요약도 함께 사라지므로, 파일로 남기고 싶으시면 명시적으로 파일 저장을 요청하셔야 합니다.

올바른 요청 방법

두 단계를 한 번에 지시하시는 게 안전합니다.

"지금까지 결정된 스키마 규칙, 수정한 파일 경로, 남은 이슈를 정리해서 PROGRESS.md 파일로 저장해줘"

이렇게 파일명과 "저장해줘"를 명시하면 클로드 코드가 실제로 파일을 생성(또는 수정)합니다.

실전 팁

- **파일명을 프로젝트마다 고정해두기:** 예를 들어 GD RDM 프로젝트라면 매번 같은 이름(PROGRESS.md)을 쓰도록 습관화하시면, 다음 세션 시작할 때 @PROGRESS.md 한 번만 멘션해도 이전 맥락을 바로 불러올 수 있습니다.
- **덮어쓰기 vs 누적:** "저장해줘"라고만 하면 클로드가 기존 파일을 덮어쓸지 이어붙일지 애매하게 판단할 수 있습니다. 의도를 분명히 하시려면 "기존 PROGRESS.md 내용은 유지하고 이번 세션 결과를 추가해줘" 또는 "최신 상태로 덮어써줘"처럼 지정하시는 게 좋습니다.
- **CLAUDE.md 와는 구분하기:** CLAUDE.md 는 세션마다 항상 자동으로 로드되는 파일이라 여기에 진행 상황을 계속 누적하면 매 턴 토큰 낭비로 이어집니다. 진행 상황 노트는 PROGRESS.md 처럼 필요할 때만 @ 멘션으로 불러오는 별도 파일로 분리하시는 걸 권장합니다.